DIGITAL SIGNAL PROCESSING

ASSIGNMENT 2

2678964R - Jude Robinson *2655002L - Yaohan Liang*

1.0 ECG Analysis

The most noticeable noise component within an ECG is the 50Hz interference, shown in Figure 1, which can be seen on the frequency domain represented by a harmonic at 50Hz. This 50Hz interference is mainly caused by the AC mains power line that gets mixed in with recording, can happen for both offline recording and real time processing.

There are two other frequency components that we might consider removing to produce a cleaner ECG, these are the bassline wander (the signal getting randomly offset from 0 at a very low, almost DC, frequency <1Hz) and any frequencies above 100Hz as the general bandwidth of the ECG is 100Hz to 150Hz and anything above are majority consider as noise or interference. These frequencies are too high to correspond to the bodily phenomena we are interest in and instead will be caused by muscle noise or electrical noise from the sensor.

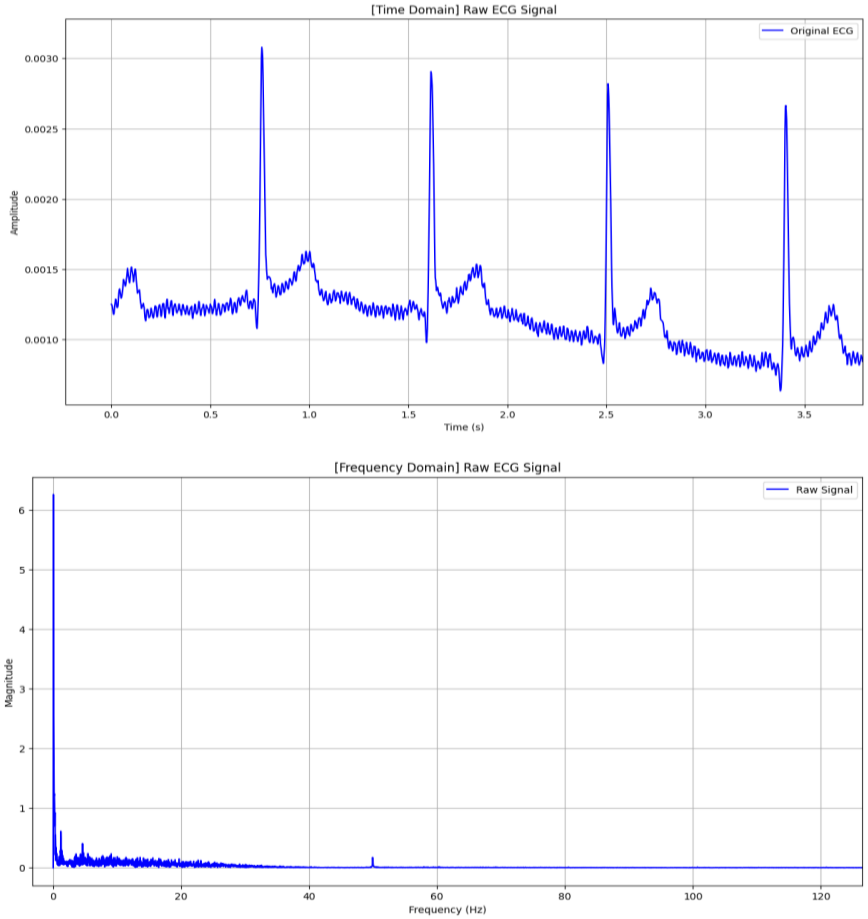


Figure 1. Raw ECG Plot (Top plot is Time domain & Bottom plot is Frequency domain) – showing the 50Hz Interference

As you can see in Figure 1, there is a lot of interference between each R-peak on the time domain, this is the >100Hz muscle and electrical noise as well as the 50Hz interference we discussed. In the

frequency domain, the largest component harmonic is around 0Hz (DC), this corresponds to the baseline wander (or DC offset) we want to remove, however we need to be careful to preserve the harmonics around 1Hz and above as these make up the useful parts of the ECG signal (60 BPM heart rate corresponds to 1Hz). In the frequency domain we can also clearly see a spike at 50Hz, again representing the mains interference which we want to remove.

As for the trade-offs, the goal was to remove as much of the unwanted noise whilst leaving the useful ECG trace intact. This is a balancing act as there will always be some level of frequency overlap between 'useful' and 'noise' frequencies, so removing one will inherently remove some of the other. The following are the rages which we experimentally found to give the best results.

For baseline wander, we ideally want to remove any frequency bellow 1Hz however we found that If we remove frequencies above 0.75Hz, it will start distorting the signal, causing the ECG to be less 'sharp'.

A similar distortion also happens if the band-stop filter around 50Hz has too large of a range. Ideally only exactly 50Hz would be removed but, since mains noise might not sit precisely on that frequency, we have to filter out a range of frequencies centred on 50Hz. We found a range of $\pm$ 1Hz (49 to 50Hz band-stop) reliably cut out the 50Hz noise.

Finally, for the low-pass filter that removes the high frequency we set the cutoff at 100Hz. Although there is noise bellow this frequency we found that lowering the cutoff at all would decrease magnitude of our R-peaks.

2.0 FIR Filter Coefficients

We have created a **firfilter** function that will create a 'rectangular' frequency domain that will remove all the frequency components (baseline wander, mains noise, and high frequency noise). We then perform an Inverse Fourier Transforms (IFFT) on this rectangular frequency domain to find the coefficients of the FIR filter.

The function has two input parameters, the sampling rate **fs** and a resolution value **delta_f**. The resolution value **delta_f** will affect how many taps there is. In the code, we are calculating the amount of **M** taps as:

$$M = \frac{fs}{\Delta f}$$

As we increase **M** taps, we get more coefficients returned by the function which will produce sharper ECG plots and efficient filtering. But the trade-off will be if **M** taps is too high, it will cause delay due to the significant amount of computation the function requires. We then set the range of frequency components we want to remove:

- Band-stop, 49Hz to 51Hz
- Baseline wander, high-pass cutoff at 0.75Hz
- Low-pass, cutoff at 100Hz

we want the band-stop to be as narrow as possible as the goal is to only remove the 50Hz interference and any more could distort the signal. Generally, it is advised to remove around 0.5Hz for the baseline wander to target wider range of types of ECG but we chose 0.75Hz as the cutoff to be a little stricter since the ECG recording is specifically of people in the 20 to 30 age range. The low-pass is 100Hz because we want to remove as much noise as possible while keeping most of the ECG. The ECG

will be interference by different noises no matter where it is recorded, so 100Hz is a good spot to filter.

Then we perform the IFFT and only taking the real frequencies, removing the complex ones. We then make the impulse response centre and symmetric, so the FIR filter has a linear phase. Finally, we apply a Hamming window function to reduce the ripples in the impulse response, and this will produce a smoother linear phase filter.

3.0 FIR Filter Implementation & Results

The **FIRfilter** class, Figure 5, wraps the FIR coefficient and provides 'real time' (sample by sample) functionality through the **.dofilter** method. In the constructor, the coefficients are stored in **self.h** and a delay line of the same length is initialized to zeros. The **dofilter** method updates the delay line by shifting the past samples, inputting the newer sample at the front and then computes the output as a dot product of the coefficients and the delay line (the same as multiplying every coefficient by it's corresponding time delayed sample). The shifting of the past samples will introduce a delay to the ECG plot.

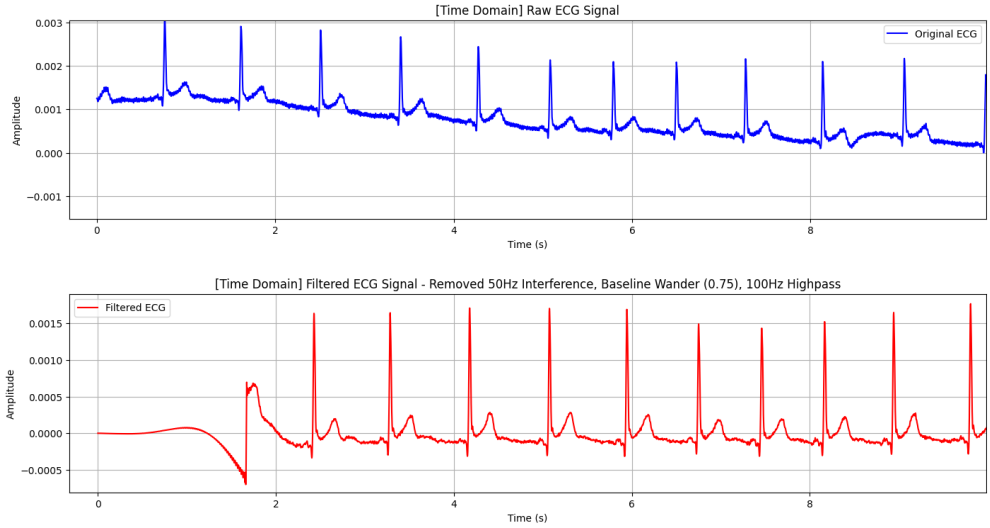


Figure 2. Time Domain plots of Raw ECG (Blue) & FIR Filtered ECG (Red)

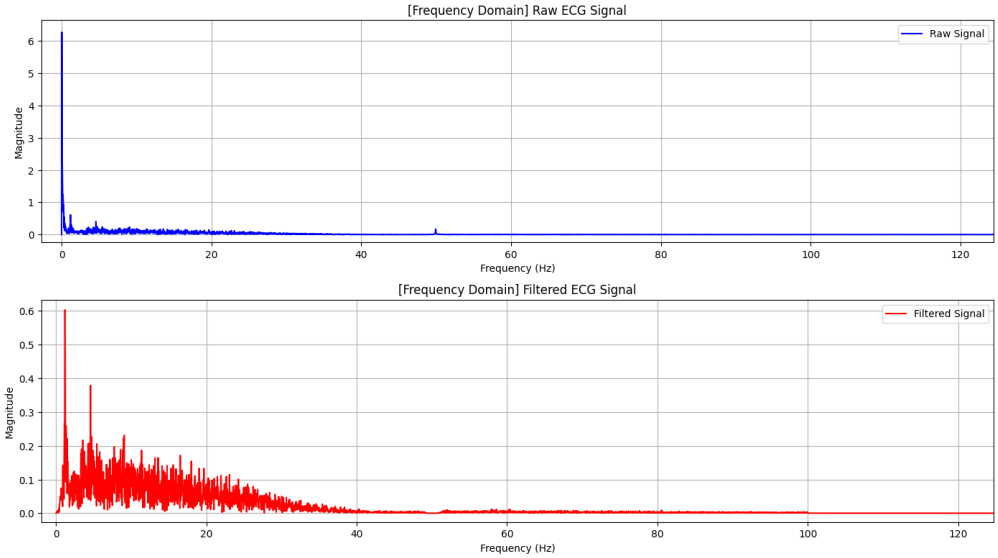


Figure 3. Frequency Domain plots of Raw ECG (Blue) & FIR Filtered ECG (Red)

Analysing the frequency domain after filtering the ECG, we can see that it has removed the large baseline DC frequency component, the harmonic at 50Hz, and any frequencies above 100Hz, just as desired. This leaves us with the frequencies of just the 'useful' part of the ECG, now a lot easier to see and analyse.

In summary, the FIR filter follows the following steps:

- The **firfilter** function will first produce a 'rectangular' frequency response from 0 to **fs/2.**
- Then we remove the frequency components for baseline (high-pass cutoff 0.75Hz), 50Hz interference (band-stop range 49Hz to 51Hz), and above 100Hz (low-pass cutoff 100Hz).
- We perform inverse Fourier transform, only keeping the real parts and adding a Hamming window function.
- The function will then produce FIR coefficients.
- Within class **FIRfilter,** the function **dofilter** will calculate the dot product with the coefficient **h** and the delayed input samples.
- The delay in the filtered plot is caused by the FIR shifting input samples through a delay line.

```
X[(f >= 49) & (f <= 51)] = 0  # Notch at 50 Hz
X[f < 0.75] = 0               # High-pass cutoff
X[f > 100] = 0                # Low-pass cutoff
```

Figure 5. Code Snippet **firfilter** function - showing the frequency component that was removed

```
class FIRfilter:

    def __init__(self, _coefficients):
        # FIR filter
        self.h = np.array(_coefficients)
        self.M = len(self.h)

        self.delay_line = np.zeros(self.M)

        # LMS adaptive filter
        self.lms_weights = np.zeros(self.M)
        self.lms_delay_line = np.zeros(self.M)

    def dofilter(self, v):
        self.delay_line[1:] = self.delay_line[:-1]
        self.delay_line[0] = v
        y = np.dot(self.h, self.delay_line)
        return y
```

Figure 6. Code Snippet **FIRfilter** class

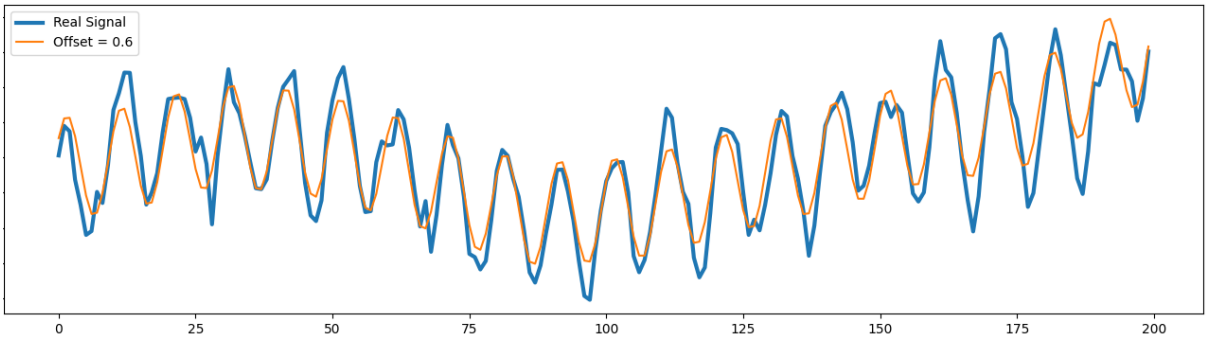
4.0 LMS Filter

Here we will implement an adaptive LMS filter that takes in a noise signal as well as the noisy ECG and iteratively improve the coefficients of the FIR filter to eliminate that noise from the signal. First, we will create an artificial 50Hz sinusoidal signal to emulate the 50Hz mains noise present in the ECG signal, then implement an adaptive LMS method in our existing FIR filter class, and finally we will use and tune this filter on our ecg data.

4.1 Generating 'Noise' signal

As previously stated, for the noise we are generating a 50Hz sinusoidal signal which could be generated 'at runtime' in our time loop. However, we decided to generate and save to an array one period of the sinusoidal noise at startup and then simply referenced this array at runtime.

To confirm the final noise signal matched the 50Hz noise in the data we offset it by a rolling average of the ECG data and plotted them together, as you can see below the signals match very closely.



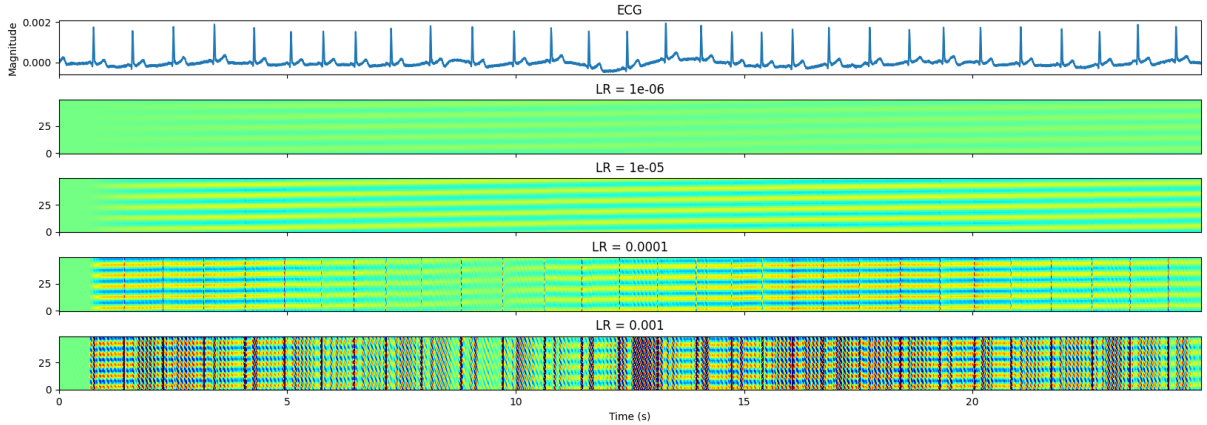
Note that offsetting the noise was purely for visualisation purposes and the pure sinusoidal signal was used for the actual filtering, the equation for which can be seen below.

```
noise = np.sin((2 * np.pi * i * freq/fs)
```

4.2 LMS method Implementation

The first step of the LMS method is calling the previously defined **dofilter** method, passing the noise sample as it's input. We can then compare the output of that to the ECG sample to get an error. For each FIR coefficient we then multiply its corresponding sample by that error and the learning rate to get how much we need to change that coefficient by. This is called stochastic gradient decent, where each of the coefficients 'moves down the slope' each iteration to minimise the square error between the filtered noise and ECG signal.

To confirm the adaptive filter was working we plotted the signal along with a colourmap plot of the filter's coefficients. This lets us easily visualise how the coefficients gradually optimise throughout the runtime. Below you can see this plot at four different learning rates, illustrating how we can increase the speed of learning and 'aggressiveness' of the filter, t the cost of incurring artifacts.

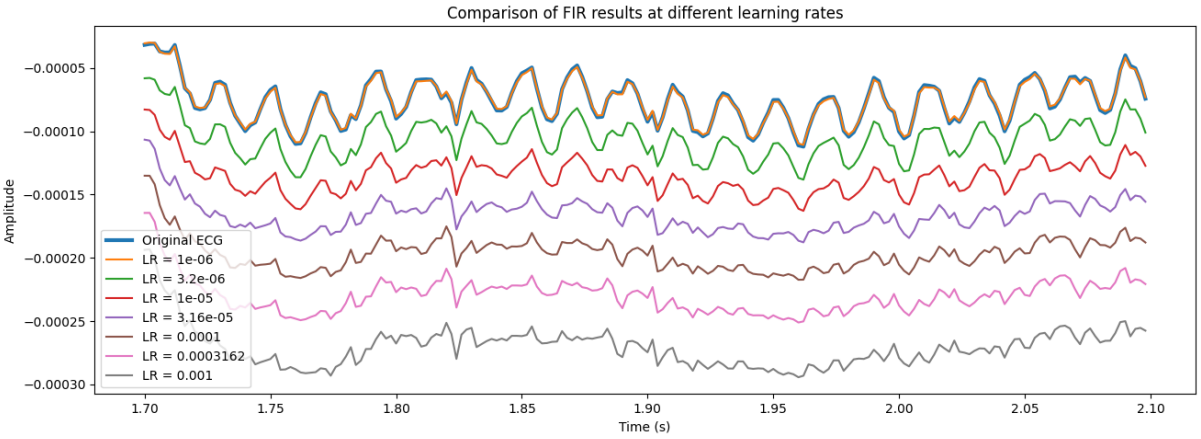


4.3 Tuning & Results

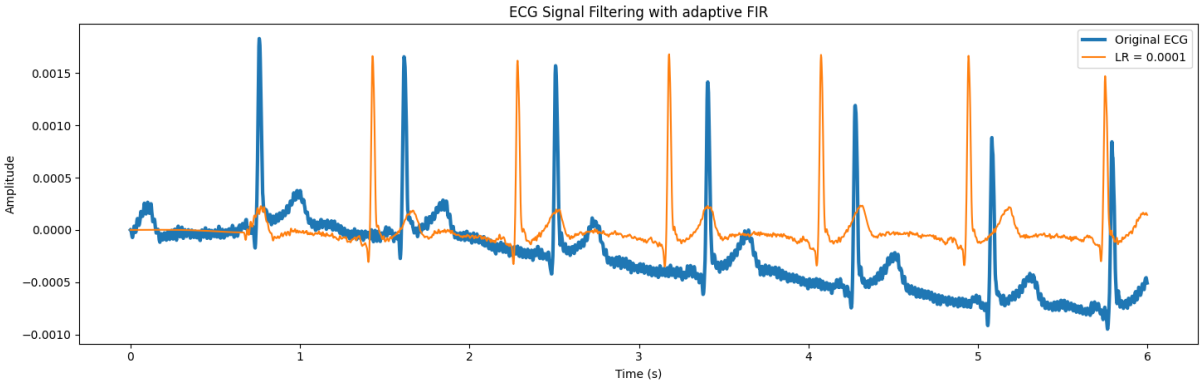
The learning rate must be tuned to get the best results out of the filter; A learning rate too low will result in the noise not being filtered out or only filtered out after a very long time, whilst a high learning rate could result in the filter being 'too reactive' and overshooting the minimum or even becoming unstable.

This can be seen in the previous plot where a learning rate of 1e-0.6 results in the filter never developing strong coefficients, whilst a learning rate of 0.001 results in highly unstable coefficients that over-react after every pulse. Whilst this is illustrative of the extreme effects of learning rate, we will optimise the learning rate based off a qualitative judgement on how the actual filtered ECG signal

looks. We did this, like with the noise, by plotting the results at a range of different learning rates. This can be seen plotted below with a small offset added to each learning rate for visual clarity and in an accidental tribute to joy division.



We have zoomed in on a 0.4 second period between pulses within the first two seconds of the filter adapting. Different learning rates are plotted from lowest to highest, top to bottom. None of the results show perfect noise elimination however a reduction in the 50Hz noise can clearly be seen as the learning rate increases. In our case a learning rate of 0.0001 seemed to effectively cut the peaks off the 50 Hz noise while not creating any major artifacts.



Finaly as you can see, we plotted the filtered against the unfiltered ECG, clearly showing that the 50Hz noise has nearly been eliminated. The main advantage of this type of LMS filter is that it leaves every other component of the signal nearly intact. A static fir filter, in contrast, must implement a band stop around 50Hz to compensate for the actual noise being slightly less or slightly more than 50Hz, but this means you could be unintentionally removing non noise components of the signals.

5.0 Matched Filter

In this section we will combine previously implemented filtering mechanisms with a matched filter to detect pulses. With these detected pulses we will then implement heuristics to validate them and finally output the instantaneous heart rate, all in ‘real time’.

5.1 Wavelet Implementation

After researching existing literature on r-pulse detection with wavelets and visual comparison we decided on a Daubechies Wavelet, specifically db4. This wavelet is the result of iterating two refinement equations with a specified set of parameters. MATLAB has a built in function for

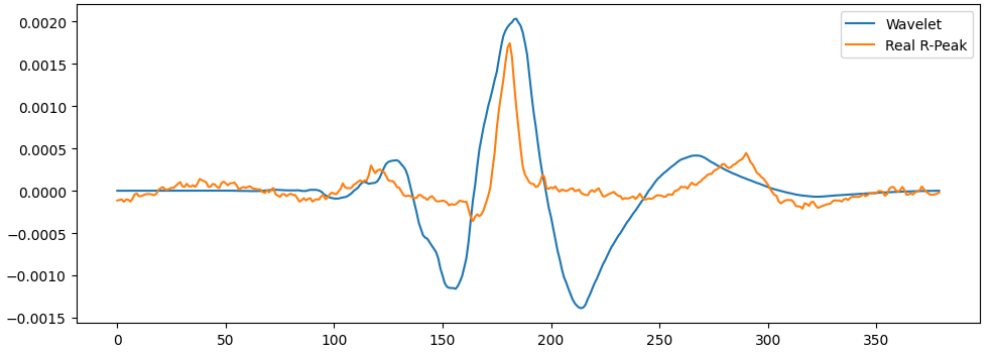
generating this wavelet which we utilised along with a down-sampling algorithm similar to the one used in assignment 1, the code for which can be seen below.

```
wavelet_name = 'db4';
iterations = 8;     % Number of iterations. Higher = smoother, more points.
[phi, psi, xval] = wavefun(wavelet_name, iterations);


% resample
numSamples = 380;
sampledPsi = zeros([1 numSamples]);
step = size(psi,2)/numSamples;
for i = 1:numSamples
    sample = psi(int64(round(i*step - step + 1)));
    sampledPsi(i) = sample;
end

writematrix(sampledPsi, 'db4.csv')
plot(sampledPsi);
```

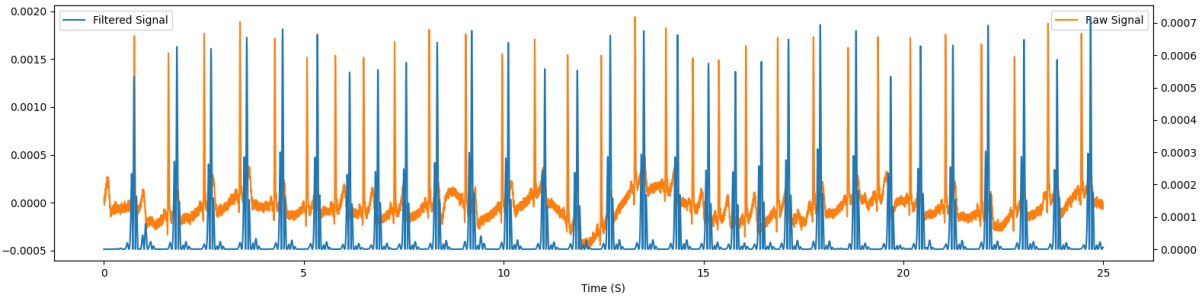
Exporting this as a csv file allows it to be imported by the python programme, making it faster and more efficient as opposed to implementing the wavelet generation within python. The wavelet plotted against a real R-pulse form the ECG can be seen plotted below.



5.2 Matched Filter implementation

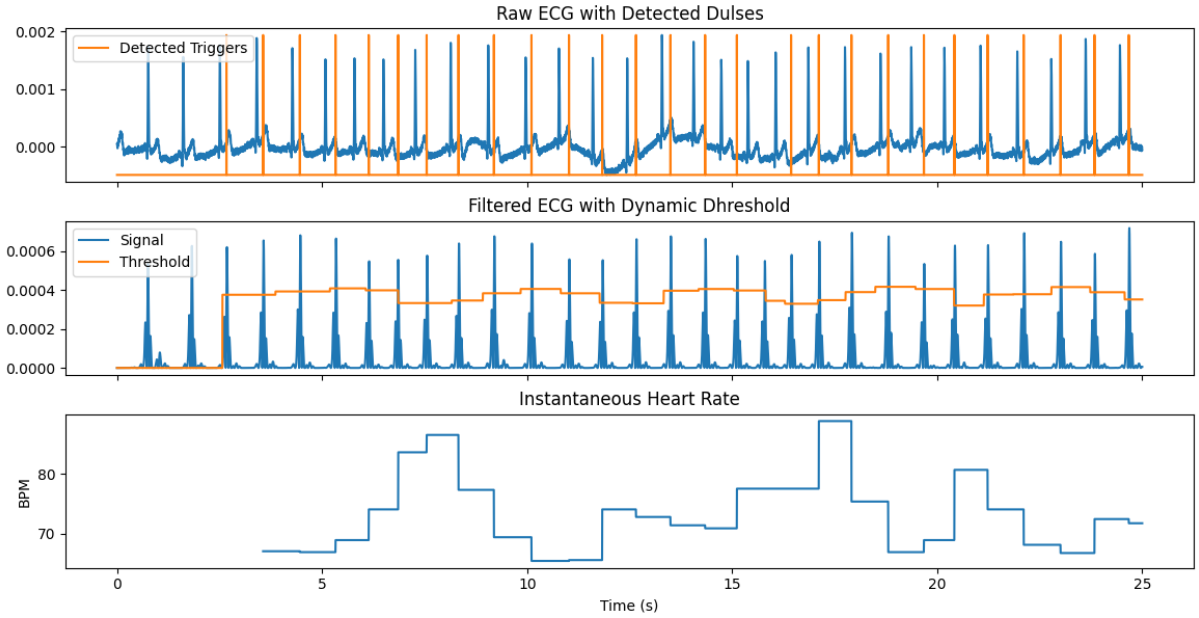
Before applying the matched filter, we eliminate as much noise as possible along with dc offset using methods previously established. This gives the matched filter a clean signal to process.

The matched filter is implemented with the exact same **FIRfilter** class however we set the coefficients to the values of our template mirrored (this is because our fir filter 'looks backwards' in time so coefficient 0 corresponds to the most recent sample). Lastly, squaring the output of our matched filter further separates the peaks of the detected pulses out from the background noise.



5.3 Pulse Detection and Heuristics

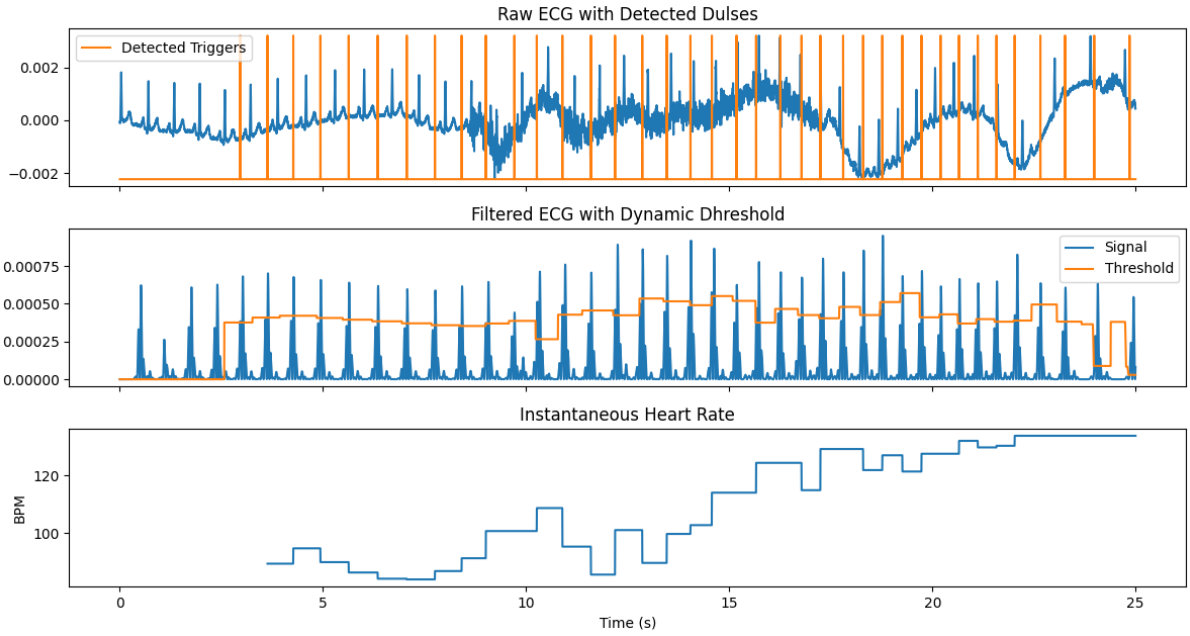
The core of our pulse detection is a simple threshold, if the filtered signal passes that threshold we count a pulse. However, since the pulses change in magnitude, we need to change the threshold to match. We do this by keeping a buffer of the past 1.5 pulses of data and finding the max value in that buffer each cycle. We then multiply this value by 0.8 to get the threshold. Note that since before we detect any pulses we don't know how big to make the buffer, we start with a pessimistic estimate of 30BPM (the longest buffer) and then update the buffer's effective length every time we get a new estimate of heartrate. One downside of this method is that we have to wait to fill the buffers (approximately 2 seconds) before we can start detecting pulses, but once they are filled, we have a highly robust and adaptable threshold. This threshold can be seen varying alongside the filtered signal in the second plot below.



Next, we implemented three heuristics to protect our system from making false readings. Firstly, I assumed a maximum and minimum heart rate of 200 and 30 BPM respectively, we then also enforced a maximum heartrate change of 15BPM per pulse. The maximum heart rate and maximum heart rate increase could be easily enforced by setting a cooldown timer on pulse detecting. This also stopped the programme from recording multiple triggers from one spike.

To enforce a minimum heart rate/ maximum heart rate decrease, each time a pulse was triggered, we checked the time since last pulse. If this time was longer than allowable according to our heuristic we would assume we missed a pulse and not update the BMP. We also had to ensure that our code could handle rare edge cases such as the second ever detected pulse being missed meaning we didn't have a BPM to default to and would have to reset the process.

To stress test this system we ran it on a worst-case scenario ECG which was normal for the first 8 seconds but which we tensed and moved around whilst taking the following 16 seconds. This results in very high muscle noise whilst tensing, and very high dc wonder whilst moving around. The results can be seen plotted on three graphs below. The first showing the raw ECG along with where our programme detected pulses. The second shows the filtered signal along with the dynamic threshold used to detect triggers, and the last shows a plot of the estimated instantaneous heart rate in BPM.



As you can see, even though we do miss a couple pulses and the accuracy does drop, the heuristics and dynamic threshold ensures the BPM stays in a realistic range and close to the real value.